

TRIE DATA STRUCTURE

A **Trie** (also called a **Prefix Tree**) is a tree-based data structure used for efficient storage and retrieval of strings. Each node represents a character, and paths from the root represent words.

Trie is widely used for prefix-based searching such as autocomplete and dictionary applications.

Rules of Trie Data Structure

1. The root node represents an empty string.
2. Each edge corresponds to a single character.
3. Each node can have multiple children but only one parent.
4. Words with common prefixes share the same path.
5. End of a word is marked using a special flag.
6. A word exists only if the last character node is marked as end of word.

Trie Node Structure

Each Trie node contains:

- An array or map of child pointers
- A boolean flag indicating end of word

Insertion Operation

Algorithm

1. Start from the root node.
2. For each character in the word:
 - If the child node exists, move to it.
 - Otherwise, create a new node.
3. Mark the last character node as end of word.

Search Operation

Algorithm

1. Start from the root node.
2. For each character of the word:
 - If the child exists, move to it.
 - Otherwise, stop and report word not found.
3. After last character:
 - If end-of-word flag is true, word is found.
 - Else, word is not found.

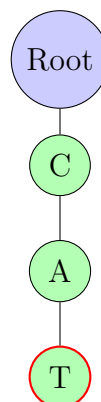
Words to Insert

CAT, CAR, DOG

Color Legend

- **Green** – Newly inserted nodes
- **Blue** – Existing nodes
- **Red border** – End of word

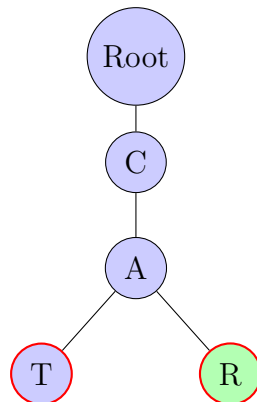
Step 1: Insert “CAT”



Explanation:

- Characters C, A, and T are inserted sequentially
- Node T is marked as end of word

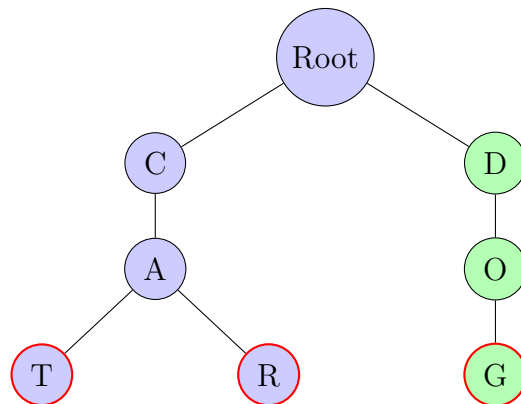
Step 2: Insert “CAR” (Tree Updated)



Explanation:

- Prefix “CA” already exists
- New node “R” is added
- “R” is marked as end of word

Step 3: Insert “DOG” (Tree Updated)



Explanation:

- New branch starting with “D” is created
- Nodes D, O, and G are inserted
- “G” is marked as end of word

Final Trie Tree

Words stored: CAT, CAR, DOG

Important Observations:

- Common prefixes share the same path
- Each word ends at a node marked as end-of-word
- Tree grows horizontally for new prefixes and vertically for characters

Time Complexity

Let L be the length of the word.

- Insertion: $O(L)$
- Search: $O(L)$

Advantages

- Efficient prefix searching
- No hash collisions
- Fast lookup time

Disadvantages

- High memory usage
- Not space efficient for small datasets

Applications

- Autocomplete systems
- Spell checking
- Dictionary implementation
- IP routing

Conclusion

Trie is an efficient data structure for storing and searching strings. Insertion and search are performed character by character in linear time, making Trie ideal for prefix-based applications.